

Day 6: Generative Modeling & DDPM

- **Generative Modeling & DDPM** — The variational view — from VAEs to diffusion
- 3 hours lecture + practical
- Slides, notes, and code on the course site

- Deep Generative Modeling
- Variational Autoencoders
- The Forward (Noising) Process
- The Reverse (Denoising) Process
- Training DDPM
- From DDPM to SDEs (Outlook)

1 · Deep Generative Modeling

1.1 The Generative Goal

- Map a **simple** distribution (Gaussian noise) to a **complex** one (data)
- Sample $x \sim p_{\text{data}}$ we can only access through examples
- Diffusion: don't jump noise $\rightarrow$ data in one step — move in many small steps
- Forward = corrupt data into noise; reverse = learn to undo it
- Source: Principles of Diffusion Models, Ch. 1–2

1.1 The Generative Goal

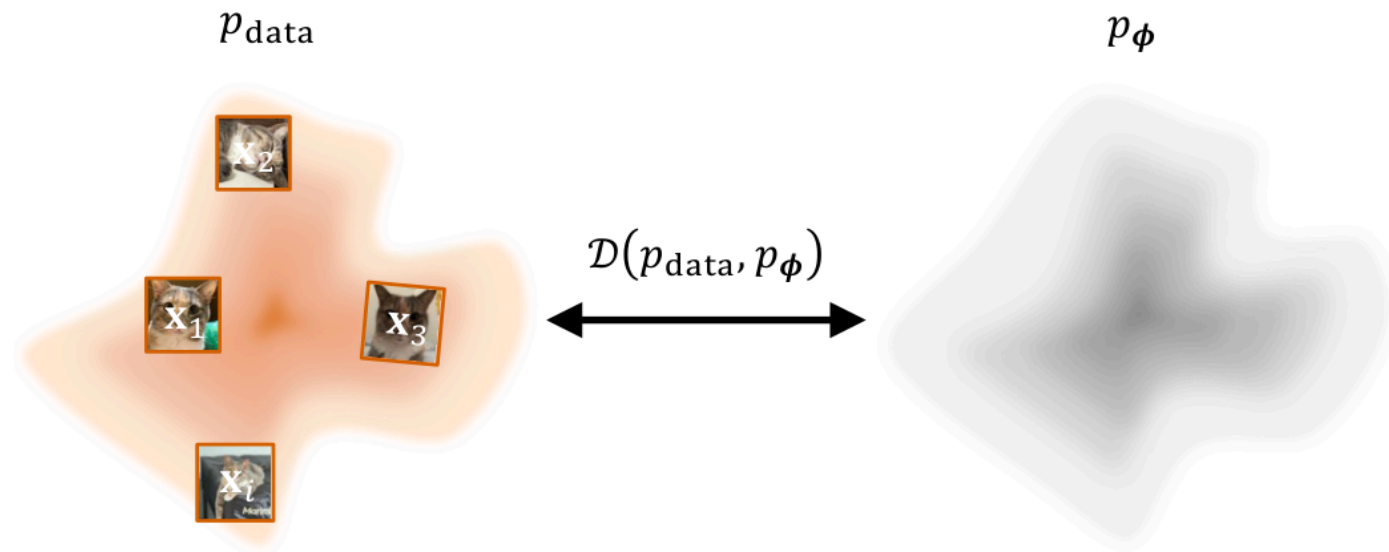


Figure 1.1: Illustration of the target in DGM. Training a DGM is essentially minimizing the discrepancy between the model distribution p_{ϕ} and the unknown data distribution p_{data} . Since p_{data} is not directly accessible, this discrepancy must be estimated efficiently using a finite set of independent and identically distributed (i.i.d.) samples, $\mathbf{x}_i$, drawn from it.

Source: Created by the author.

1.2 Taxonomy of Models

- Likelihood-based: VAEs, normalizing flows, autoregressive, diffusion
- Implicit: GANs (no tractable $p(x)$)
- Trade-offs: sample quality vs exact likelihood vs sampling speed
- Diffusion = likelihood-based + high quality, but slow sampling

1.2 Taxonomy of Models

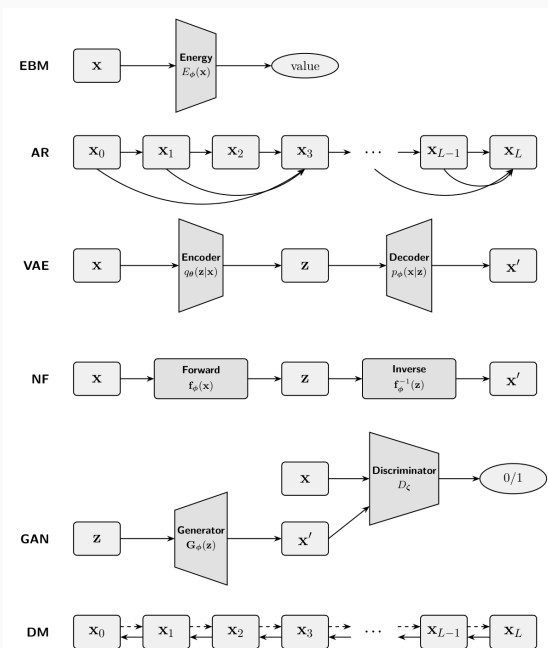


Figure 1.2: Computation graphs of prominent deep generative models. Top to bottom: **EBM** maps an input $\mathbf{x}$ to a scalar energy; **AR** generates a sequence $\{\mathbf{x}_t\}_{t=0}^L$ left to right with causal dependencies; **VAE** encodes $\mathbf{x}$ to a latent $\mathbf{z}$ and decodes to a reconstruction $\mathbf{x}'$; **NF** applies an invertible map f_θ between $\mathbf{x}$ and $\mathbf{z}$ and uses f_θ^{-1} to produce $\mathbf{x}'$; **GAN** transforms noise $\mathbf{z}$ to a sample $\mathbf{x}'$ that is judged against real $\mathbf{x}$ by a discriminator D_θ ; **DM** iteratively refines a noisy sample through a multi-step denoising chain $\{\mathbf{x}_t\}_{t=0}^L$. Boxes denote variables, trapezoids are learnable networks, ovals are scalars; arrows indicate computation flow. The trapezoid was intended to indicate a dimension-changing transformation, while rectangles denote variables or mappings that preserve dimensionality.

1.3 Latent-Variable Models

- Introduce latent z : $p_{\theta}(x) = \int p_{\theta}(x|z)p(z) \mathrm{d}z$
- Prior $p(z) = N(0, I)$ is easy to sample
- Marginal likelihood is intractable (integral over all z)
- Posterior $p_{\theta}(z|x)$ also intractable $\rightarrow$ variational inference

2 · Variational Autoencoders



2.1 The Evidence Lower Bound

- Encoder $q_\varphi(z|x)$ approximates the true posterior
- $\log p_\theta(x) \geq \mathbb{E}_{q_\varphi}[\log p_\theta(x|z)] - D_{\text{KL}}(q_\varphi(z|x) \parallel p(z))$
- Reconstruction term + regularization term
- Gap to $\log p(x)$ is exactly $D_{\text{KL}}(q_\varphi(z|x) \parallel p_\theta(z|x)) \geq 0$

2.1 The Evidence Lower Bound

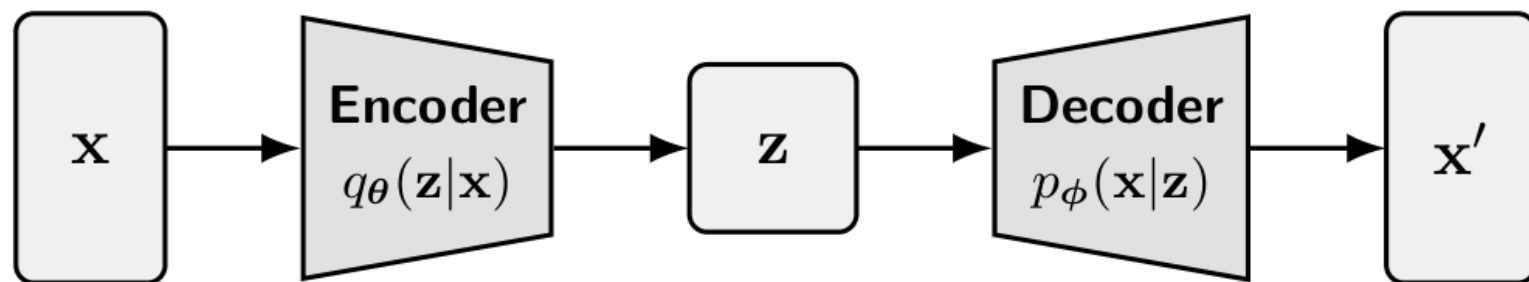


Figure 2.1: Illustration of a VAE. It consists of a stochastic encoder $q_{\theta}(\mathbf{z}|\mathbf{x})$ that maps data $\mathbf{x}$ to a latent variable $\mathbf{z}$, and a decoder $p_{\phi}(\mathbf{x}|\mathbf{z})$ that reconstructs data from the latent.

Source: Created by the authors

2.2 Derivation: ELBO in Three Lines

- Start: $\log p_\theta(x) = \log \int p_\theta(x, z) \, dz$
- Multiply & divide by q_φ : $\log \mathbb{E}_{q_\varphi} \left[p_\theta(x, z) \frac{1}{q_\varphi}(z|x) \right]$
- Jensen (log concave): $\geq \mathbb{E}_{q_\varphi} \left[\log \left(p_\theta(x, z) \frac{1}{q_\varphi}(z|x) \right) \right]$
- Split the log: $= \mathbb{E}_{q_\varphi} [\log p_\theta(x|z)] - D_{\text{KL}}(q_\varphi \parallel p(z))$
- Exact gap: $\log p(x) - \text{ELBO} = D_{\text{KL}}(q_\varphi(z|x) \parallel p_\theta(z|x)) \geq 0$
- Same algebra reused for the diffusion ELBO below

2.3 Reparameterization Trick

- Sample $z = \mu_{\varphi}(x) + \sigma_{\varphi}(x) \cdot \varepsilon, \varepsilon \sim N(0, I)$
- Moves randomness off the computation path $\rightarrow$ low-variance gradients
- Backprop flows through $\mu_{\varphi}, \sigma_{\varphi}$
- DDPM = a **deep hierarchy** of these latents with a fixed encoder

3 · The Forward (Noising) Process

3.1 Unified Forward Rule

- $x_t = \alpha_t x_0 + \sigma_t \varepsilon, \varepsilon \sim N(0, I)$
- $p(x_t \mid x_0) = N(x_t; \alpha_t x_0, \sigma_t^2 I)$
- α_t = signal kept; σ_t = noise added
- Signal-to-noise ratio $\text{SNR}(t) = \frac{\alpha_t^2}{\sigma_t^2}$ decreases with t

3.1 Unified Forward Rule

$$\mathbf{x}_0 \sim p_{\text{data}}$$

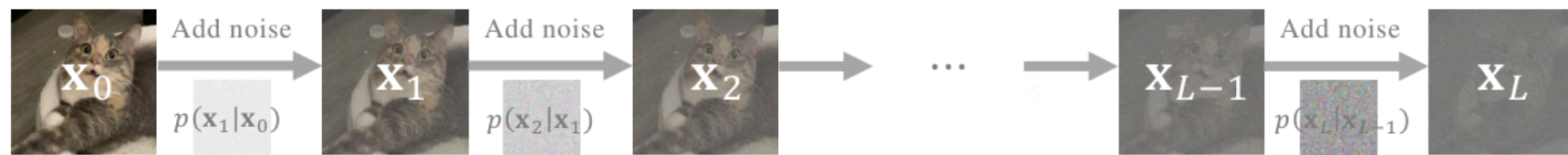


Figure 2.4: Illustration of the DDPM forward process, wherein Gaussian noise is incrementally added to corrupt a data sample into pure noise.

3.2 Closed-Form Marginal

- Step kernel $q(x_t \mid x_{t-1}) = N(\sqrt{1 - \beta_t}x_{t-1}, \beta_t I)$
- Compose Gaussians: $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\varepsilon$
- $\bar{\alpha}_t = \prod_{s=1}^t (1 - \beta_s)$ so $\alpha_t = \sqrt{\bar{\alpha}_t}$
- Sample any t in one shot — no need to simulate the chain

3.3 Derivation: Compose the Gaussians

- Write $a_t = 1 - \beta_t$, so $x_t = \sqrt{a_t}x_{t-1} + \sqrt{1 - a_t}\varepsilon_t$
- Substitute one step into the next (telescoping)
- Two independent Gaussians add $\rightarrow$ variances add
- $a_t(1 - a_{t-1}) + (1 - a_t) = 1 - a_t a_{t-1}$
- Iterate to x_0 : $x_t = \sqrt{\bar{a}_t}x_0 + \sqrt{1 - \bar{a}_t}\varepsilon$
- VP schedule: $\alpha_t^2 + \sigma_t^2 = 1$

3.4 Noise Schedules

- VP (DDPM/cosine): $\alpha_t^2 + \sigma_t^2 = 1$
- VE (EDM): $\alpha_t = 1, \sigma_t = t$
- Linear interpolation (flow matching): $\alpha_t = 1 - t, \sigma_t = t$
- Interactive — compare schedules on the same image (see notes)

3.4 Noise Schedules

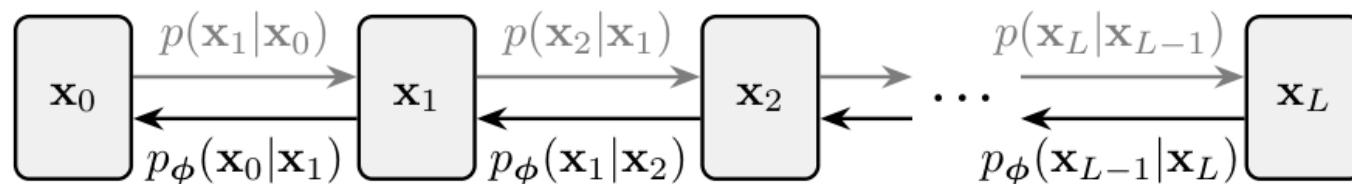


Figure 2.3: Illustration of DDPM. It consists of a fixed forward process (in gray) that gradually adds Gaussian noise to the data, and a learned reverse process that denoises step-by-step to generate new samples.

Source: Created by the author.

4 · The Reverse (Denoising) Process

4.1 Reverse as Denoising

- Start at $x_T \sim N(0, I)$, walk back to data
- Learn $p_\theta(x_{t-1} \mid x_t)$ — one denoising step
- Oracle reverse kernel $p(x_{t-1} \mid x_t)$ is intractable...
- ...but $q(x_{t-1} \mid x_t, x_0)$ **is** Gaussian (condition on x_0)

4.1 Reverse as Denoising

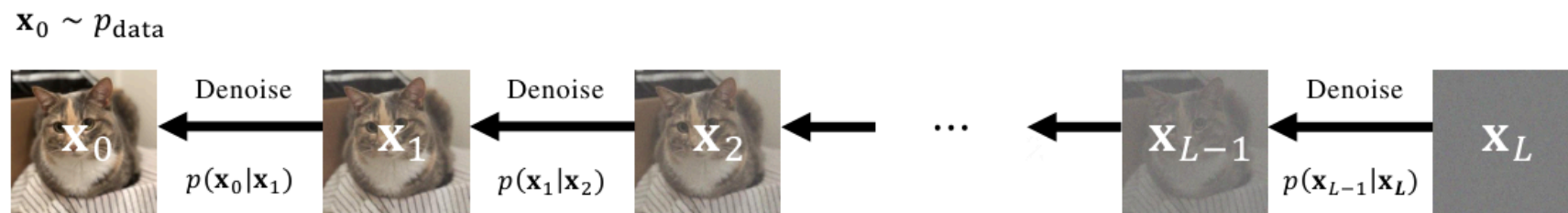
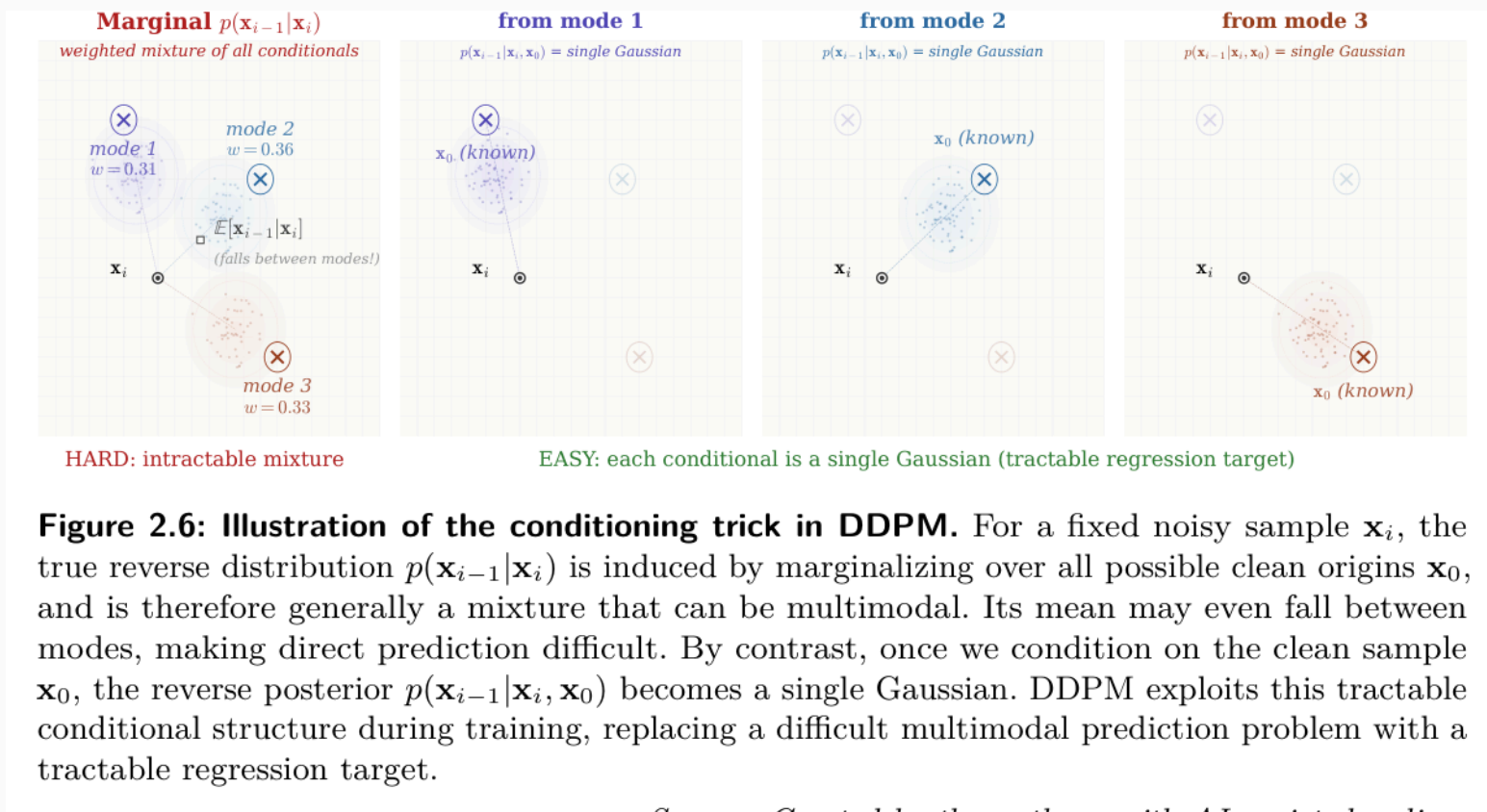


Figure 2.5: Illustration of DDPM reverse (denoising) process. Starting from noise $\mathbf{x}_L \sim p_{\text{prior}}$, the model sequentially samples $\mathbf{x}_{i-1} \sim p(\mathbf{x}_{i-1}|\mathbf{x}_i)$ for $i = L, \dots, 1$ to obtain a newly generated data $\mathbf{x}_0$. The oracle transition $p(\mathbf{x}_{i-1}|\mathbf{x}_i)$ is unknown; thus, we aim to approximate it.

4.2 The True Posterior

- $q(x_{t-1} \mid x_t, x_0) = N(\bar{\mu}_t(x_t, x_0), \bar{\beta}_t I)$
- $\bar{\mu}_t = \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1-\bar{\alpha}_t}x_0 + \frac{\sqrt{1-\beta_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}x_t$
- $\bar{\beta}_t = \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\beta_t$
- Conditioning trick: the secret sauce that makes training tractable

4.2 The True Posterior



4.3 Derivation: Gaussian Posterior

- Bayes: $q(x_{t-1}|x_t, x_0) \propto q(x_t|x_{t-1})q(x_{t-1}|x_0)$
- Both factors Gaussian $\rightarrow$ collect quadratic & linear terms in x_{t-1}
- Precision: $\frac{1}{\bar{\beta}_t} = a_t \frac{1}{\beta_t} + \frac{1}{1-\bar{\alpha}_{t-1}}$
- Variance: $\bar{\beta}_t = (1 - \bar{\alpha}_{t-1}) \frac{1}{1-\bar{\alpha}_t} \beta_t$
- Mean: $\bar{\mu}_t = (\sqrt{\bar{\alpha}_{t-1}} \beta_t) \frac{1}{1-\bar{\alpha}_t} x_0 + (\sqrt{a_t}(1 - \bar{\alpha}_{t-1})) \frac{1}{1-\bar{\alpha}_t} x_t$

4.4 Parameterizing the Reverse Step

- Match $p_\theta(x_{t-1}|x_t) = N(\mu_\theta(x_t, t), \bar{\beta}_t I)$ to the posterior
- Predict x_0 , the noise ε , or the velocity v — equivalent
- ε -prediction: μ_θ written via $\varepsilon_\theta(x_t, t)$
- Network sees (x_t, t) only — never the true x_0

5 · Training DDPM

5.1 Variational Bound to Simple Loss

- ELBO over the chain = $L_T + \sum_t L_{t-1} + L_0$
- Each $L_{t-1} = D_{\text{KL}}(q(x_{t-1}|x_t, x_0) \parallel p_\theta(x_{t-1}|x_t))$
- KL of two Gaussians = weighted $\|\bar{\mu}_t - \mu_\theta\|^2$
- Substitute ε -parameterization $\rightarrow$ clean noise-prediction loss

5.2 Derivation: KL to Noise Prediction

- Same-covariance Gaussians: $L_{t-1} = 1 \frac{1}{2\beta_t} \|\bar{\mu}_t - \mu_\theta\|^2$
- Estimate $\hat{x}_0 = \left(x_t - \sqrt{1 - \bar{\alpha}_t} \varepsilon_\theta\right) \frac{1}{\sqrt{\bar{\alpha}_t}}$
- Substitute into $\bar{\mu}_t$ and μ_θ
- Prefactors $1 \frac{1}{\sqrt{a_t}}$ and x_t cancel — only noises remain
- $L_{t-1} \propto \|\varepsilon - \varepsilon_\theta(x_t, t)\|^2$
- Set the weight to 1 $\rightarrow L_{\text{simple}}$ (up-weights hard, high-noise steps)

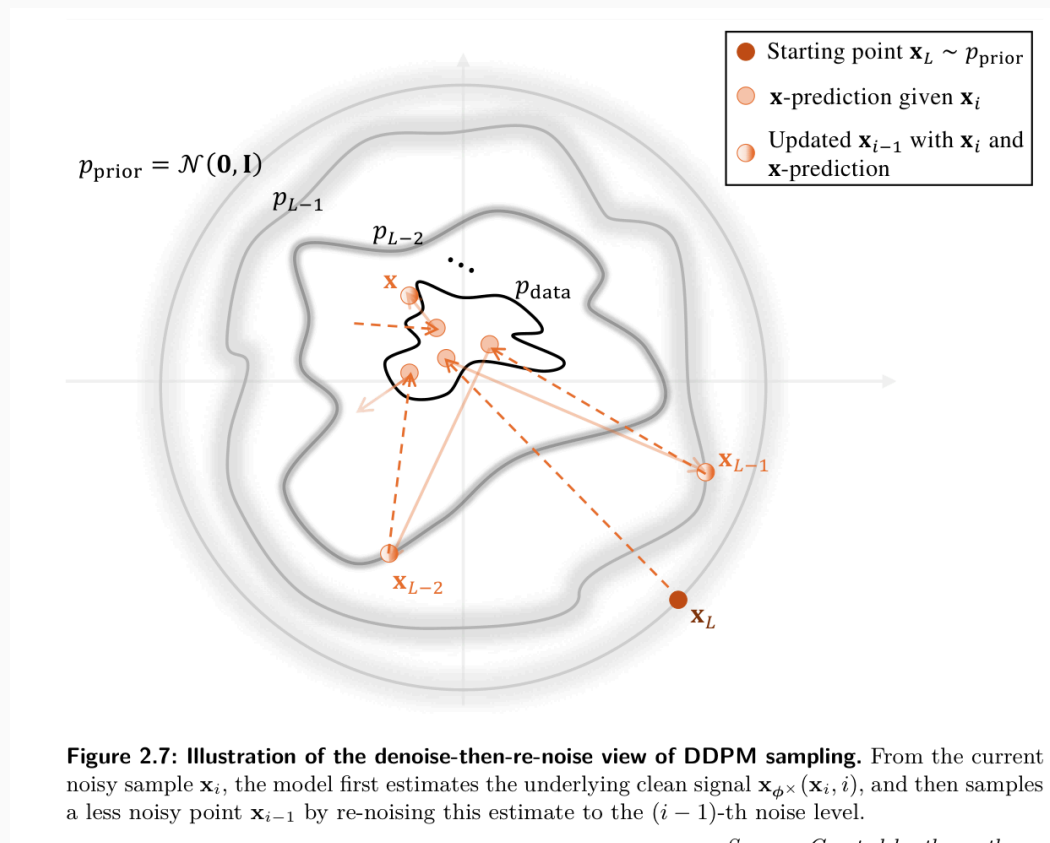
5.3 The ε -Prediction Objective

- $L_{\text{simple}} = \mathbb{E}_{t, x_0, \varepsilon} [\|\varepsilon - \varepsilon_{\theta}(x_t, t)\|^2]$
- $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon$
- Drops the awkward per- t weights — works better in practice
- Tweedie links ε to the score: $\nabla \log p_t(x_t) = -\varepsilon_{\theta} \frac{\backslash}{\sigma_t}$ (Day 7)

5.4 Sampling: Denoise then Re-noise

- From x_t : predict ε_θ , estimate $\hat{x}_0$, jump to posterior mean
- $x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \beta_t \frac{1}{\sqrt{1-\bar{\alpha}_t}} \varepsilon_\theta \right) + \sqrt{\beta_t} z$
- Add a little fresh noise $z \sim N(0, I)$ (except at the last step)
- Repeat $T \rightarrow 0$ — this is ancestral sampling
- Day 8: do this in far fewer steps with ODE/SDE solvers

5.4 Sampling: Denoise then Re-noise



6 · From DDPM to SDEs (Outlook)

6.1 Forward & Reverse as SDEs

- Take the step size to zero: the chain becomes a continuous-time SDE
- Forward: $dx = f(x, t) dt + g(t) dw$ (drift + Brownian noise)
- Reverse: $dx = [f - g^2 \nabla \log p_t(x)] dt + g d\bar{w}$
- The score $\nabla \log p_t(x)$ replaces the unknown reverse drift
- Probability-flow ODE shares the same marginals (deterministic sampling)
- DDPM is the discretized variance-preserving SDE — full derivations in the SDE course

6.1 Forward & Reverse as SDEs

Training: learn score $\mathbf{s}_t(x) := \nabla \log p_t(x)$ by

1. Sample from target $x_1 \sim p_1$
2. Run "forward" SDE to "noise" x_1 until it becomes "simple" $x_0 \sim p_0$
3. Minimize some loss $\propto \|\hat{\mathbf{s}}_t(x) - \mathbf{s}_t(x)\|^2$

Inference: sample using "reverse" SDE or prob-flow ODE

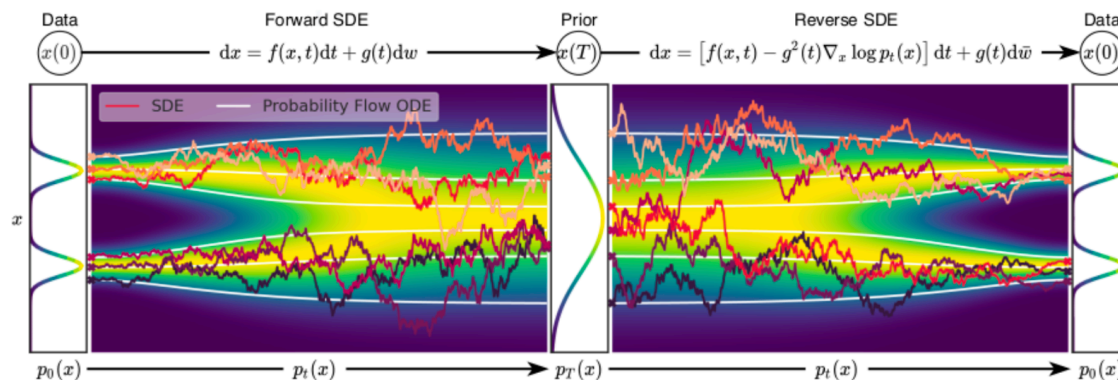


Figure 2 from Song et al. (2020)

6.2 Simulating SDEs: Euler-Maruyama

- Discretize $[t_0, t]$ into steps of size Δt
- Brownian increment $\Delta\beta(t_k) \sim N(0, \Delta t I)$
- $x_{k+1} = x_k + b(x_k, t_k)\Delta t + \sigma(x_k, t_k)\Delta\beta(t_k)$
- Noise enters at scale $\sqrt{\Delta t}$ since $\text{Var} = \Delta t$
- DDPM ancestral sampling = the VP special case of this scheme
- Reference: SDE course (Brownian motion, time reversal, Girsanov)

6.2 Simulating SDEs: Euler-Maruyama

Algorithm 1: Euler-Mayurama Discretisation

input: $[t_0, t]$, $p(\mathbf{x}(t_0))$, Δt , $d\mathbf{x}(t) = \mathbf{b}(\mathbf{x}(t), t) + \boldsymbol{\sigma}(\mathbf{x}(t), t)d\boldsymbol{\beta}(t)$

- 1 Divide the interval $[t_0, t]$ into steps of size Δt : $t_0, t_0 + \Delta t, \dots, t_0 + k\Delta t, \dots, t$
 - 2 $\mathbf{x}(t_0) \sim p(\mathbf{x}(t_0))$
 - 3 **for** *each step* k **do**
 - 4 $\Delta\boldsymbol{\beta}(t_k) \sim \mathcal{N}(\mathbf{0}, \Delta t \mathbb{I}_d)$
 - 5 $\mathbf{x}(t_{k+1}) = \mathbf{x}(t_k) + \mathbf{b}(\mathbf{x}(t_k), t_k)\Delta t + \boldsymbol{\sigma}(\mathbf{x}(t_k), t_k)\Delta\boldsymbol{\beta}(t_k)$
 - 6 **end**
 - 7 **return** $\{\mathbf{x}(t_k)\}_k$
-

- Day 6: **Generative Modeling & DDPM**
- The variational view — from VAEs to diffusion
- Questions welcome — practical follows

Questions?

Practical session follows — see course site.